

NYCU-ECE DCS-2026

HW05

Design: Simple CPU

Data Preparation

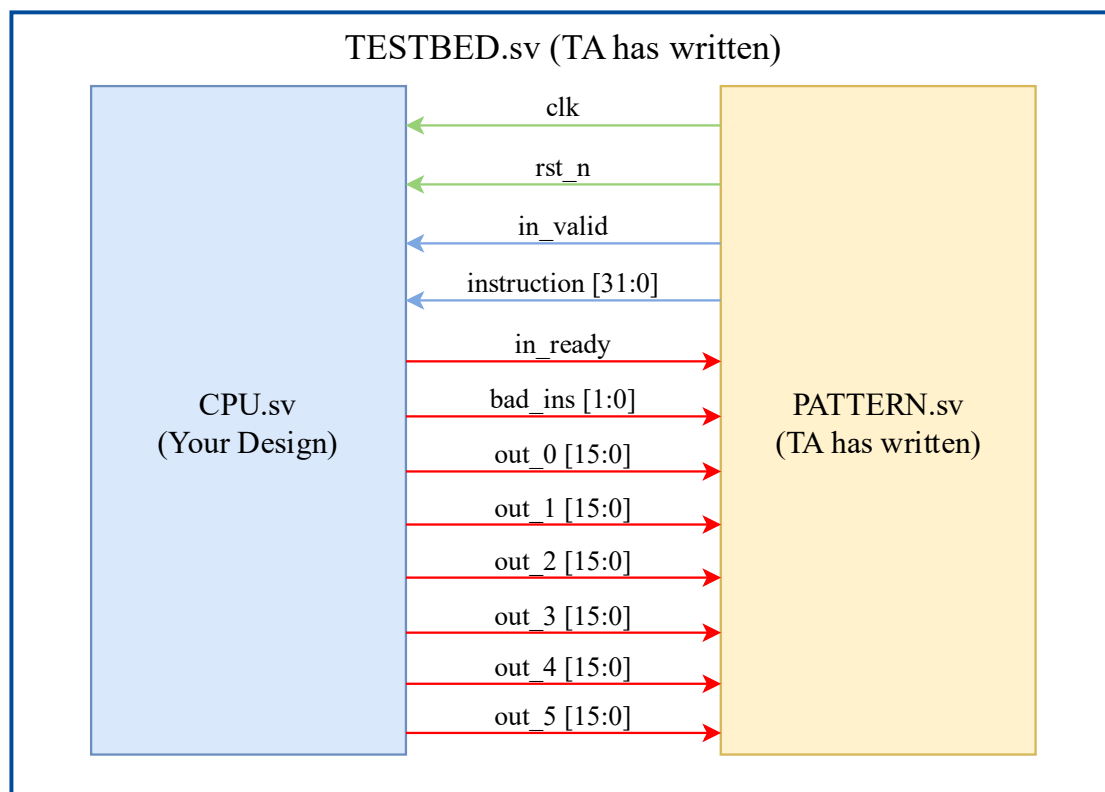
1. 從 TA 目錄資料夾解壓縮:

```
$ tar -xvf ~es26dcsTA01/HW05.tar
```

2. 解壓縮資料夾 HW05 包含以下:

```
HW05/  
├── 00_TESTBED/  
├── 01_RTL/  
├── 02_SYN/  
├── 03_GATE/  
└── 09_UPLOAD/
```

Block Diagram



Design Description

這次作業需要設計一個簡單的CPU，並根據輸入的指令(32-bit)，判斷需要做哪些運算，更新相對應暫存器的值，並輸出結果。

➤ 指令格式

類型	位																															
	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	opcode (6)						rs (5)					rt (5)					rd (5)					shamt (5)					funct (6)					
I	opcode (6)						rs (5)					rt (5)					immediate (16)															

■ R-type 運算指令

*D[rx] 指位址為 rx 之 register 中存的 data

Opcode(6)	Funct(6)	Operation	Description
000000	100000	ADD	$D[rd] = D[rs] + D[rt]$
000000	011000	MULT	$D[rd] = (D[rs] * D[rt]) \ggg 15$
000000	011001	OR	$D[rd] = D[rs] \mid D[rt]$
000000	000000	SLA	$D[rd] = D[rt] \ll \text{shamt bits}$
000000	000010	SRA	$D[rd] = D[rt] \ggg \text{shamt bits}$
000000	110001	DIV	$D[rd] = (D[rs] \ggg n) / D[rt]$

*n 為一正整數，計算方法稍後會說明

■ I-type 運算指令

Opcode(6)	operation	description
001000	ADDI	$D[rt] = D[rs] + \text{immediate}$
001101	ORI	$D[rt] = D[rs] \mid \text{immediate}$

➤ 資料型態

本次 design CPU 的暫存器以及 immediate 的 data type 是 16-bit **signed fixed-point**，除了第一個 bit 為 sign bit 之外，其餘 15 bits 為 fraction bits。

S	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
---	----	----	----	----	----	---	---	---	---	---	---	---	---	---	---

S 代表 -2^0 ，14 ~ 0 分別代表 $2^{-1} \sim 2^{-15}$ 。注意，本次 fixed point format 採用的是 2's complement，詳見以下範例：

$$0_1100000000000000 = 2^{-1} + 2^{-2} = 0.5 + 0.25 = 0.75$$

$$1_1100000000000000 = -2^0 + 2^{-1} + 2^{-2} = -1 + 0.5 + 0.25 = -0.25$$

此格式的數學有效範圍是： $[-1.0, 0.99996948]$ ，解析度為 2^{-15} (約等於 0.0000305)，零用十六進位表示為 16'h0000。因為表示範圍有限，針對容易超過表示範圍的乘除運算 (Mult / Div) 需做特殊處理，詳見 **R-type 運算指令及運算詳細說明**。

➤ 運算詳細說明

1. 乘法指令先將兩個 16-bit signed fixed-point 數視為 signed integer 相乘，得到 32-bit signed 結果，再對該 32-bit 結果做 arithmetic right shift 15 bits，取低 16 bits 寫回 rd。(相當於算完乘法後 $\ggg 15$ ，如同上方表格所標示)
2. 除法指令的商須準確到小數點後第15位。被除數在運算前視情況須進行 n bits 的右移，以確保被除數小於除數，讓商能夠維持在本次的 signed fixed-point 表示範圍內。n 的計算方法如下：

假設由 rs, rt 分別取出 A, B 兩數，考慮以下兩種情況：

- ◆ 若 $|A| < |B|$ ，則 A 不需右移， $n = 0$ 。
 - ◆ 若 $|A| \geq |B|$ ，則為了讓被除數小於除數，須將 A 右移。找出 A, B 中從 MSB (不含 sign bit) 往下數遇到第一個 1 的位置，分別為 Pos_A 和 Pos_B (範圍 0~14)， $n = Pos_A - Pos_B + 1$ 。
 - ◆ 延續上點敘述，若 $A = -1$ ， $Pos_A = 15$ 。
3. 加法若發生溢位不須特別處理，維持正常 2's complement 的 wrap-around 行為即可。
 4. 所有指令皆須將運算結果寫回指定的暫存器。
 5. 若指令合法且成功執行，bad_ins = 2'b00。
 6. 若收到不符合格式的指令、無效的 register address，輸出結果時將 bad_ins 設為 2'b01。

7. 若除法指令 D[rt] 值為 0，無須執行該次指令，該次指令不更新任何 register，輸出結果時將 bad_ins 設為 2'b10。
8. 若同時發生多個 bad_ins，以 2'b01 優先判斷。
9. Left shift / Right shift 指令中 rs 不參與運算，但仍須為有效 register address。
10. shamt, immediate 即為 value，可直接做計算。
11. Instruction 的 rs, rt, rd 為 address，上述運算須讀取 register 的 value (D[rx]) 出來運算。共有6個 register，同學需在自己的 design 中宣告這6個 register，並分別對應下表的六個 Address 及初始 Value。

Address(5 bits)	Value(初始為0)
10001	0
10010	0
01000	0
10111	0
11111	0
10000	0

➤ 運算範例

Instruction : 001000-10001-10010-0100000000000000

I-type Addi: Opcode-rs-rt-imm

假設 $\text{reg}(10001) = 0$ ， $\text{imm} = 0.5 \Rightarrow \text{reg}(10010) = 0 + 0.5 = 0.5$

Instruction : 001100-10001-10010-0000000000000100

There's no such opcode as 001100 $\Rightarrow$ set bad_ins to 2'b01

Instruction : 000000-10001-10010-10111-00001-000000

R-type Sla: Opcode-rs-rt-rd-shamt-funct

假設 $\text{reg}(10010) = 0.25$, $\text{shamt} = 1 \Rightarrow \text{reg}(10111) = 0.25 \ll 1 = 0.5$

Instruction : 000000-10001-10010-10111-00001-110001

R-type Div: Opcode-rs-rt-rd-shamt-funct

假設 $\text{reg}(10001) = 1_0000_0010_0110_001$ (約 -0.99069)

$\text{reg}(10010) = 0_0000_1000_0000_001$ (約 0.03128)

$|\text{reg}(10001)| = \sim(0000_0010_0110_001) + 1 = 1111_1101_1001_111$

$|\text{reg}(10010)| = 0000_1000_0000_001$

$(Pos_A, Pos_B) = (14, 10), n = 14 - 10 + 1 = 5$

$\text{reg}(10111) = (\text{reg}(10001) \ggg 5) / \text{reg}(10010)$

$= -(0_0000_0111_1110_110 / 0_0000_1000_0000_001)$

$= -111_1110_1010_0000 = 1_000_0001_0110_0000$ (約 -0.98926)

➤ 測資行為

■ 輸入讀檔: input.txt

第一行為測資總數量，第二行開始為 instruction。

```
1 5000
2 00000010001010001000111111000010
3 00000010001010001011101101000000
4 00000010001100011001010000000000
5 0000001111110010100000000110001
```

00_TESTBED 中附上與之對應的組合語言 input_readable.txt 供參考

■ 輸出讀檔: output.txt

由左到右分別為 bad_ins, out_0, out_1, out_2, out_3, out_4, out_5。

若bad_ins非00，則 out_0, out_1, out_2, out_3, out_4, out_5 輸出當前對應的暫存器的值。

1	00	0	0	0	0	0	0
2	00	0	0	0	0	0	0
3	00	0	0	0	0	0	0
4	10	0	0	0	0	0	0
5	00	0	0	0	-19569	0	0
6	00	0	0	0	-19569	0	0
7	10	0	0	0	-19569	0	0
8	00	0	0	0	-19569	0	0
9	00	0	0	0	-19569	0	0
10	00	0	0	0	-19569	0	0

■ Read after Write 情況

助教的 demo 測資會排除任意連續 4 個 instruction 出現 read after write data dependency 的情形，例如以下情況：

instruction 1 -> **r1** = r2 + r3

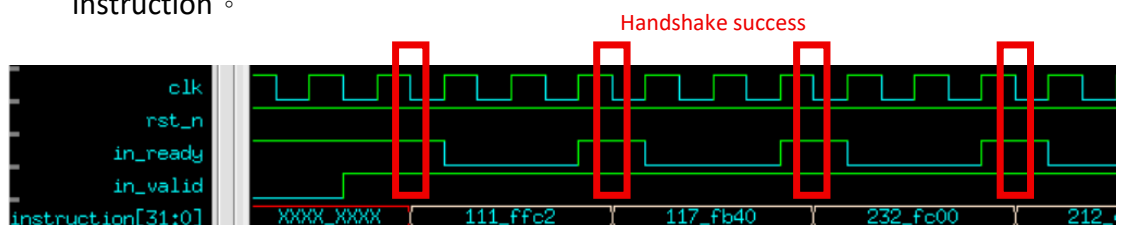
instruction 2 -> r4 = r2 + r4

instruction 3 -> r4 = r3 + r4

instruction 4 -> r4 = **r1** + r4

■ PATTERN 行為

- ◆ 在 asynchronous active-low reset 後，所有的 output 訊號必須全部歸零。
- ◆ 除了 in_ready 以外，在 out_valid 為 0 時其餘所有 output 訊號須維持 0。
- ◆ PATTERN 在 in_valid 拉起之後的每個 clock negedge 檢查 in_ready 是否為 1 (handshake detection)。若是，則提供下一條指令；若否，則維持當前輸出。
- ◆ 換言之，本次作業是由同學自行控制輸入時序，in_ready 可以連續抬起。In_valid 在抬起後會維持抬起的狀態直到送出所有 instruction。



- ◆ 每條指令應該要對應到一次 `out_valid` 以及其餘輸出，不允許在該次 `in_valid/in_ready` handshake 前就拉起 `out_valid`，`out_valid` 最快只能在對應的 handshake 結束後第一個 clock posedge 拉起。

- ◆ `in_ready` 必須在對應的 `in_valid` 拉起後 10 個 cycle 內拉起。也就是說，同學的 design 向 PATTERN 請求 instruction 的頻率不能太慢，每 10 個 cycle 至少要請求一筆 instruction。
- ◆ `out_valid` 要在對應的 `in_valid` & `in_ready` 成功握手後 10 個 cycle 內拉起，可連續抬起。
- ◆ Latency 的計算方法為 `in_valid` 拉起到最後一筆 `out_valid` 拉起所花的 cycle 數。

Inputs

Signal name	Number of bit	Description
clk	1 bit	clock
rst_n	1 bit	Asynchronous active-low reset
in_valid	1 bit	當 PATTERN 拉起 <code>in_valid</code> 時可以開始進行 handshake 來取得 instruction；當 <code>in_valid</code> 掉下時代表已經沒有更多 instruction
instruction	32 bits	當 <code>in_valid == 1</code> 且 <code>in_ready == 1</code> 時，PATTERN 送出一筆 instruction

Outputs

Signal name	Number of bit	Description
in_ready	1 bit	為 1 時向 PATTERN 請求 instruction
out_valid	1 bit	為 1 時輸出值
bad_ins	2 bit	opcode 不符格式時為 2'b01 (優先判斷) 除法指令錯誤時為 2'b10
out_0	16 bits	暫存器 10001 的值
out_1	16 bits	暫存器 10010 的值
out_2	16 bits	暫存器 01000 的值
out_3	16 bits	暫存器 10111 的值
out_4	16 bits	暫存器 11111 的值
out_5	16 bits	暫存器 10000 的值

Specifications

1. Top module name: **CPU** (File name : **CPU.sv**)
2. 請根據 **Design Description** 段落的描述，使用 `systemverilog` 來完成這次功課的 RTL code。

- 請勿使用 `*error*`, `*latch*`, `*congratulation*` or `*fail*` 當作 logic / wire / reg / submodule / parameter 的名稱，否則 demo 結果會是 fail。
Note: *代表在該 word 前後的任何符號，比如: `first_error_test` 即為禁止的。
- 本次作業禁止在 RTL code 中使用 for loop / while loop。
- 02_SYN result 不行有 error 且不能有任何 latch，且 slack result 必須要 MET。
- Input delay = 0.5 * clock period, Output delay = 0.5 * clock period。
- 03_Gate 不能有 timing violation。Latency 需與 01_RTL 模擬結果相同。
- 本次作業同學可以自行調整 clock cycle time，最小精度為小數點後一位 (0.1ns)。跑模擬時請於以下位置更改 cycle time:

```
`define CYCLE_TIME 10.0 (00_TESTBED / PATTERN.sv: line 1)
```

```
set CYCLE 10.0 (02_SYN / syn.tcl: line 11)
```

- 本次作業 Demo 會有 hidden case pattern。

Example waveform



Upload file

- 在 09_UPLOAD/ 目錄下使用 `./01_upload` 指令上傳你的 RTL design。
- 將你的報告上傳至 E3，上傳前請將檔名改為 `hw5_report_dcsXXX.pdf`。
- 因上傳腳本與 NTP 伺服器同步需要時間，所以請同學注意繳交期限，切勿在過於接近期限的時間上傳檔案。
- 1de 繳交期限: **2026/05/19 15:30:00**; 2de 繳交期限: **2026/05/26 15:30:00**。
(請同學注意繳交期限，與前幾次功課稍有不同)
- 檔名不符合規範者扣 5 分

Grading policy

1. Pass the RTL-Level simulation & Synthesis & Gate-Level simulation: 60%
2. Performance: 30%
Ranking formula: $(\textit{Latency} \times \textit{clockperiod})^2 \times \textit{area}$ (less is better)
3. Report: 10%
4. For submission in 2nd demo, total score will get **40% off**

Note

參考指令:

1. 01_RTL/ (RTL simulation) → **./01_run_vcs_rtl**
2. 02_SYN/ (synthesis) → **./01_run_dc**
3. 03_GATE/ (gate-level simulation) → **./01_run_vcs_gate**
4. 09_UPLOAD/ (upload) → **./01_upload**